Docker Basic Commands

A hands-on lab cheat sheet using the Nginx image

This guide walks through the essential Docker commands in the order you would use them in a first lab: finding an image, downloading it, running it in different modes, inspecting it and cleaning up.

Based on the OpenWriteup tutorial: https://www.openwriteup.com/?page_id=801

Quick Reference

Command	Purpose
`docker search`	Find images on Docker Hub
`docker pull`	Download an image
`docker images`	List local images
`docker run`	Create and start a container
`docker run -it`	Run with an interactive shell
`docker run -d`	Run in the background
`docker ps / ps -a`	List running / all containers
`docker stop`	Stop a running container
`docker rename`	Rename a container
`docker restart`	Restart a container
`docker exec -it`	Run a command in a running container
`docker logs`	View container output
`docker rm`	Remove a stopped container

Step-by-Step Commands

1. Search for an image

```
docker search nginx
```

Queries Docker Hub and lists images related to the keyword. Use it to discover what is published before you download anything.

2. Pull an image

```
docker pull nginx
```

Downloads the official Nginx image from Docker Hub to your machine. With no tag, the latest version is fetched. To get a particular version, add a tag after the name, for example nginx:1.25.

3. List local images

```
docker images
```

Shows every image stored locally, with its repository, tag, ID, creation time and size.

4. Run a container (foreground)

```
docker run nginx
```

Creates and starts a container from the image. By default it runs in the foreground, so your terminal stays attached to the container output and is blocked until the container stops. Press Ctrl+C to exit.

> Generic form: docker run <image name>

5. Run a container interactively

```
docker run -it nginx /bin/bash
```

Starts a new container and opens an interactive bash shell inside it instead of the default Nginx process. Type exit to leave the shell.

> -i keeps input open and -t allocates a terminal; together they give you a usable shell.

6. Run a container in the background

```
docker run -d nginx
```

Detached mode. The container runs in the background and your terminal is free to use for other commands.

7. List running containers

```
docker ps
```

Displays the containers that are currently running, with their ID, image, status, ports and name.

8. Stop a container

```
docker stop <container id / container name>
```

Gracefully stops a running container. You can identify it by either its ID or its name.

9. Rename a container

```
docker rename <old name> <new name>
```

Changes the name of an existing container without recreating it.

10. Restart a container

```
docker restart <container id>
```

Stops and starts the container again.

11. Run a command inside a running container

```
docker exec -it mynginx bash
```

Executes a command in a container that is already running, here opening a bash shell. Give the container name or ID.

> Unlike docker run, exec does not create a new container.

12. View container logs

```
docker logs <container id / container name>
```

Prints the output a container has written, which is the first place to look when something misbehaves.

13. List all containers (including stopped)

```
docker ps -a
```

Shows both running and stopped containers along with their details.

14. Remove a container

```
docker rm <container id / container name>
```

Deletes a stopped container. Stop it first with docker stop.

Docker Hub Account

To push your own images or pull private ones you need a free Docker Hub account. Sign up at https://hub.docker.com/signup.

Typical Lab Flow

```
docker search nginx docker pull nginx docker run -d --name mynginx nginx
docker ps docker logs mynginx docker exec -it mynginx bash docker stop
mynginx docker rm mynginx
```

> The **`--name mynginx`** flag is an addition to the original lab. It gives the container a predictable name so later commands such as exec, logs and stop can refer to it directly.